Server Latency Monitor

Description

An online campus service monitors the response latency of recent requests. Log records arrive in chronological order. Each record contains a timestamp and a response latency. After processing each record, report the highest latency within the last W seconds, the ID of the corresponding record, and whether the alert threshold H has been reached.

There are n records, numbered from 1 to n in input order. Record i is (t_i, d_i): t_i is its timestamp in seconds, and d_i is its response latency in milliseconds. Timestamps are nondecreasing; multiple records may have the same timestamp.

Monitoring rules

1. When processing record i, consider only records already read: $1 \le j \le i$. A later record must not be included yet, even if it has the same timestamp.

2. The active window is $(t_i - W, t_i]$. Record j is active exactly when $t_i - W < t_j \le t_i$. A record at the left endpoint is excluded. The current record is always active.

3. Let M_i be the maximum latency among active records. If several records have latency M_i, let k_i be the smallest record ID among them.

4. Output `ALERT` if $M_i \ge H$; otherwise, `output OK`. H is a configurable software threshold supplied in the input.

Input

The first line contains three integers n, W, and H. Each of the next n lines contains two integers t_i and d_i describing record i. The input contains one test case.

Output

Print n lines. Line i must contain M_i, k_i, and the status, separated by single spaces. Use the uppercase status `ALERT` or `OK`. Do not print any additional text.

Constraints

$1 \le n \le 2 \times 10^5$; $1 \le W, H \le 10^9$.

$0 \le t_i, d_i \le 10^9$; $t_1 \le t_2 \le \cdots \le t_n$.

Examples

Each `output` is determined immediately after the corresponding record is processed.

Example 1

Input	Output
10 5 200 0 120 1 220 1 220 3 180 5 160 6 190 7 260 9 260 10 140 12 200	120 1 OK 220 2 ALERT 220 2 ALERT 220 2 ALERT 220 2 ALERT 190 6 OK 260 7 ALERT 260 7 ALERT 260 7 ALERT 260 8 ALERT

Records 2 and 3 both have latency 220, so record 2 wins the tie. At record 6, the window is $(1,6]$: both records at time 1 have expired, and the maximum becomes 190. At record 10, the window is $(7,12]$: record 7 has expired, so record 8 supplies the maximum of 260.

Example 2

Input	Output
6 1 100 0 100 0 100 1 80 1 90 4 100 5 0	100 1 ALERT 100 1 ALERT 80 3 OK 90 4 OK 100 5 ALERT 0 6 OK

This example covers sequential processing at equal timestamps, the excluded left endpoint, time gaps, and equality with the threshold. Record 3 cannot use record 4 yet. Record 5 has latency exactly H and triggers an alert. At record 6, the record at time 4 has expired, so the `output` is 0 6 `OK`.

Implementation requirement

Submit a complete program. With the given input size, consider the total cost of rescanning the entire window after every record. No specific library is required; any implementation that follows the rules and passes the tests is acceptable.